

Università della Calabria

Dipartimento di Matematica e Informatica



Corso di Laurea Magistrale in Informatica

Tesi di Laurea

PABEL: Plugin ABE for LLM

**Ensuring Document Confidentiality for AI Agents via
Attribute-Based Encryption**

Relatori	Candidato
Ch.mo Prof. Giovambattista Ianni	Claudio Lucisano
Dott. Ing. Andrea Altarui	Matricola 231871

Anno Accademico 2025/2026

Indice

1	Introduction	3
1.1	Context and Motivations	3
1.2	Activities done and Results	3
1.3	Technologies and Tools	4
1.4	Thesis outline	5
2	Motivation and Threat Model	6
2.1	Problem Statement	6
2.1.1	Limitations of Homomorphic Encryption	7
2.1.2	Limitations of Trusted Execution Environments	8
2.2	Actors and Goals	9
2.3	Assumptions	9
2.4	Adversary Model	10
2.5	Security Goals	11
3	Background	12
3.1	Attribute-Based Encryption	12
3.1.1	From Identity-Based to Attribute-Based Encryption	12
3.1.2	Ciphertext-Policy versus Key-Policy ABE	13
3.1.3	Multi-authority ABE	15
3.1.4	Functional Encryption as a Generalization	15
3.2	Identity and Access Management	15
3.2.1	OAuth 2.0 and PKCE	15
3.2.2	Keycloak	15
3.3	The Model Context Protocol	15
3.4	Hook-Based Interception in AI Coding Agents	15
4	System Design	16
4.1	Architecture Overview	16
4.2	Early Prototyping: File Viewer and .abe format	16
4.2.1	File Structure	16
4.2.2	File Viewer	18

4.3	CP-ABE Authority and Key Management	19
4.4	Human Identity via Keycloak	19
4.5	Agent Identity Model	19
4.6	Audit Logging	19
4.7	Core and Strategy Pattern	20
4.7.1	The Relay Hook	20
4.7.2	Per-Agent Adapters	20
4.8	Document Lifecycle	20
4.9	Deployment Architecture	20
5	Security Analysis	21
6	Evaluation	22
7	Conclusions	23
8	Bibliography	24

1 Introduction

1.1 Context and Motivations

The constant and rapid evolution in the field of Artificial Intelligence has result in a day-to-day use of AI agents, thus the need of security and privacy when dealing with tools that, potentially, could harvest personal or company data for a variety of reasons. More in detail the concept that we were trying to focus on, was the scenario in which an employee, by chance or intentionally, had its AI agent to look at a confidential file which the company doesn't want to be leaked or simply does not trust the AI agent's vendor.

This particular fear, to not trust AI vendors, it's not unfounded; several recent incidents show how sensible data can end up outside the boundaries an organization intended to enforce, or how AI providers themselves may proceed to process user data in ways that raise legitimate concerns.

A first well known example is the 2023 Samsung incident, where some employees pasted proprietary source code into a public LLM chatbot for debugging purposes, effectively transmitting confidential IP (intellectual property) to an external, unintended infrastructure. This incident led Samsung to ban all AI tools altogether[1]

A second, more institutional, example involves OpenAI: in december 2024, the Italian Data Protection Authority fined OpenAI for €15 million, finding that ChatGPT was processing users' personal data to train underlying models without the legal basis required by the GDPR; not to mention a security breach, that took place in march 2023, which exposed users' data and payment information[2].

Also Anthropic was acting in regard of the guarantees that have to be assumed by a vendor, in particular it had to update its customer terms[3]. They explicitly stated that each conversation made with Claude could be used for model training, with the exception of Claude for Work, Claude Enterprise, Claude Gov and any API-based access[3].

1.2 Activities done and Results

In this thesis, we have designed and implemented a plugin for AI agents that allows to decrypt files using Attribute-Based Encryption (ABE) in order to ensure that only authorized users and authorized AI Agents can access specific parts of the file. The plugin is designed to be easily integrated into existing AI agents, providing a seamless way to

enhance security and privacy in AI-driven environments. Such security enhancement is achieved by encrypting files with ABE, which allows for fine-grained access control based on user and AI agent attributes. The plugin intercepts file access requests from the AI agent, authenticates the user and checks, via server, if both the user's attributes and the agent's attributes match the encryption policy; thus the server decrypts the specific parts of the file that both the user and *that* agent are authorized to see and returns it to the agent. This approach ensures that sensitive information remains protected when accessed by AI agents.

1.3 Technologies and Tools

To achieve the goals of this thesis, several technologies and tools have been utilized. The core of the security mechanism is based on Attribute-Based Encryption (ABE), more in particular we have used the open source library **OpenABE**[4], which we had to upgrade to a more recent version by updating its dependencies and fixing some bugs that were present in the original code.

Another important component of the system is the authentication and authorization mechanism, which is implemented using **Keycloak**, an open-source identity and access management solution. Keycloak provides a robust framework for managing user identities, roles, and permissions, which is essential for enforcing access control policies in the context of ABE.

The plugin is designed to be compatible with AI agents that support plugin integration, and it has been tested with specific AI agents to ensure proper functionality. The AI agents used for testing are:

- **Claude Code**, an AI agent developed by Anthropic;
- **Codex**, an AI agent developed by OpenAI;
- **Copilot for VSCode**, an AI agent developed by Microsoft, integrated into the Visual Studio Code IDE.

Throughout the project we used Python as main language, as well as Python libraries such as **PyJWT**, **FastMCP** and **Psycopp**. The server-side component of the system was containerized using **Rancher Desktop**, a tool that allows to run Kubernetes clusters on

a local machine. This setup enables easy deployment and management of the server component, ensuring scalability and reliability in handling decryption requests from AI agents. Alongside the server container, there were deployed two other containers: one for the Keycloak server and another for a **PostgreSQL** database, which is used by Keycloak to store user and role information. This containerized approach simplifies the deployment process and allows for easy scaling of the system as needed.

1.4 Thesis outline

After a description of the motivation and also a security overlay of this work in section 2, we'll cover some background informations in section 3 before heading in a in-depth analysis of the System Design in section 4.

2 Motivation and Threat Model

2.1 Problem Statement

The main problem that this thesis aims to solve comes from a consideration while thinking how AI can be implemented in a blue team scenario. Even if this topic could be an argument of discussion by itself, many perplexities arise when we think how data is exchanged considering AI agents actively inside the attack surface.

In particular, the problem is that AI agents are not able to distinguish between sensitive and non-sensitive data, and they are not able to understand the context of the data they are processing. This means that if an AI agent is given access to sensitive data, it could potentially leak that data to an attacker, either intentionally or unintentionally. Of course, it is imperative to also consider the intrinsic vulnerability of AI agents to prompt injection attacks [5], which could be just another mean to the exfiltration of sensitive data as aforementioned.

One method to mitigate this problem would be to utilize cryptography, but that alone comes with its own set of challenges. More in detail, both an AI agent and a user cannot benefit from data that is encrypted with a symmetric key: it is easy to see how both of them would require a decryption of said data, otherwise they would not be able even to understand it, let alone use it.

One possible alternative could be Homomorphic Encryption and, surely, it is a very promising topic, but as of today its computational limitations make it not suitable for a real-world scenario, especially if we consider the amount of data that an AI agent should process.

Another approach to achieve more security in a scenario that involves a LLM could be Trusted Execution Environments, but here lies the need (thus the problem) to ensure that the AI agent is running in a trusted environment, which is not always possible, especially if we consider that the AI agent could be running on a remote server or in a cloud environment.

Hereby we'll expand on the limitations of both Homomorphic Encryption and Trusted Execution Environments, to better understand and define the problem we are trying to solve.

2.1.1 Limitations of Homomorphic Encryption

As aforementioned, Homomorphic Encryption is a very promising topic, given the ability it enables to perform homomorphic operations on encrypted data. The problems arise when we approach statistical computations, which is the basis of Large Language Models, and, in general, of any AI agent.

Precisely on this regard, Cheon et al. eloquently stated [6]:

Most of real-world data contain some errors from their true values. [...] an approximate value may substitute the original data and a small rounding error does not have too much effect on computation result [...] Unfortunately this rounding operation has been considered difficult to perform on HE since it is not simply represented as a small-degree polynomial.

They proposed a scheme that allows to perform approximate computations on encrypted data, not without open problems, like the need to convert said scheme in a fully homomorphic scheme using bootstrapping. This problem was later addressed by De Castro et al. who proposed a GPU-based workflow by extending the capabilities of OpenFHE [8].

Said implementations is undoubtedly a great step forward towards practical encrypted LLM, but it has still non trivial limitations:

Although promising given the massive overheads involved in both LLMs and FHE, this is still far from practical for real-world usage, even for applications that are not latency sensitive such as text summarization or content generation (in contrast to chatbots or Q/A tasks, which are more demanding in terms of responsiveness).

The main problem is that, even with a GPU-accelerated implementation, the bootstrap overhead (also in combination with the LLM inference overhead) is still too high to be considered practical in a every day scenario. To this we have to take in account also that by approximating the functions needed to have LLM inference we are introducing a source of error that translates into a loss of accuracy in the final result. Both of this factors alone make Homomorphic Encryption, as per today, not viable to be used in application such as chatbot or coding agents and it is clear why Attribute-Based Encryption could fill the gap in this point and time by providing security precisely where the FHE, at the moment, cannot provide.

2.1.2 Limitations of Trusted Execution Environments

Trusted Execution Environments, as the name suggests, are trusted environment where it is possible to execute critical operation and, in the context of prompt injection and LLM, potentially separate a network or an entire device from the execution of malicious actions by the agent that, in fact, would only be relegated in a sandbox.

Geppert et al. define TEEs as following:

TEEs [...] are defined as tamper-resistant processing environments that run on a separation kernel. Remote verification makes the technology suitable for cloud environments. It is a mechanism proving to an organization that uses the services of a cloud provider that the specified software runs on the predefined hardware even if the organization does not have direct access to this hardware.

In the cloud computing scenario this sort of infrastructure is already offered by providers like Google, Amazon, and Microsoft [9], but this would not apply to some organization that don't want to rely on external provider. This is extremely crucial even when talking of LLMs, since if an organization opted to rely on third party agents (i.e. Claude Code, Codex, etc...) it would rely to this third party also all the security assumptions that arise.

To better explain this concept, by considering the use of a TEE provided by some provider, an organization would not have any control on *how* its own data are manipulated. It is obvious that in most cases this is not an acceptable risk and many would resort to a "home-made" solution, like training its own LLM; this issue, although, could be mitigated if we consider a situation where only pre-approved data can be "sent out", thus the application of Attribute-Based Encryption and the not suitability of TEE in every scenario.

Summarizing, even though TEEs «minimizes potential security issues such as data leakage, unauthorized access, code tampering, and software-based attacks» ([10]), they do not take in consideration the cases in which a company does want to fine grain the data that escape the company's domain.

2.2 Actors and Goals

In this section we define the actors involved in the system and their goals, to better understand the problem we are trying to solve.

The actors, with their own goals, involved in the system are:

- **User:** the user is the person that wants to use an AI agent to process a document. The user has a set of attributes that define their access rights to the document.
- **AI Agent:** the AI agent is the entity that retrieves and processes the document. The AI agent has a set of attributes that define its access rights to the document.
- **Company:** the company is the entity that owns the document and defines the access policy for it alongside the attributes of the users and the AI agents. The company is responsible for ensuring that the access policy is enforced and that the sensitive data is protected.
- **IT/Cybersec:** the company's departments responsible for the security of the documents (i.e. encrypting, authorizing and auditing) and the company altogether

2.3 Assumptions

We assume that the documents subject to protection are already produced and encrypted via Attribute-Based Encryption before being made available to the system described in this work. Authoring a new document, defining its access policy, ensuring the freshness of employees' attributes, generating the corresponding keys, encrypting the document, are all separate concerns and fall out of the scope of this thesis, which focuses on enforcing, at read time, a mechanism that allows an AI agent to process the document following a safe workflow, minimizing the risk of sensitive data exfiltration.

We also assume that our guarantees end at the moment decrypted content is legitimately handed to the requesting AI agent, in accordance with the CP-ABE policy and the combined attributes of the user and the agent. What the agent does with that content afterwards is not something an access-control layer operating at application level can control, and is therefore out of scope. There is an exception worth to mention, that is locally materialized copies which are time-bounded and removed automatically at the end of a session, to limit possible unbounded exposure.

By design, the system assumes that the encrypted documents are .abe files, that is they follow a predefined structure (as defined in Section 4.2.1) that permits the recognition of encrypted files without the need to decrypt them.

2.4 Adversary Model

In this section we'll describe what entities are considered as adversary. By default, we assume the worst case scenario: both AI agent and its Vendor are not trusted, nor the user itself. Mainly, we can differentiate three different threat entities:

- **AI agent / Vendor:** the agent itself can be "too curious", it can be compromised or worse, its vendor can actively try to use customer data for its own goals (see 1.1). It's important to assume the worst case scenario, and to don't trust the model.
- **User:** both by error and intentionally, a user could try to use its AI agent to read something outside the user's authorization level.
- **Attacker:** an external entity that could also intercept traffic data and, possibly, forge malicious content.

It is imperative to understand and prevent possible data leaks, hence our proposal to put at risk only a certain part of confidential data (chosen by the company or who possess them).

It's also important to note that there are goals outside this work's scope such as:

- **OS/Endpoint already compromised:** if an attacker has already access to the machine, it is quite easy to see how this system cannot ensure its premises.
- **Secure storage of CP-ABE master key:** by definition, it is a secret and the company should implement the appropriate measures to store it safely.
- **Agent behaviour post reception of confidential data:** it is impossible for this system to control what an agent will do to a file once received. After the reception of the file, adequately decrypted, it is responsibility of the user what will happen to the data.
- **DoS attacks:** this system does not guarantee integrity, but the adequate confidentiality of the data.

2.5 Security Goals

The end goal of this work is to provide a simple way to handle sensitive data with a fine-grained access control enforced by a company, such that its own employee could not provide leaks whenever they use an AI agent, since the access control mentioned above. This allow also to audit each and every operations on encrypted files, that is offering an invaluable tool for Cybersec/IT departments, which could easily track any eventual misbehaviour.

Our work certainly benefit corporate environments, but also any enviornment where you want to limit what agents can see and therefore where more controlled access is needed, whit a consequent help on data leak control.

3 Background

3.1 Attribute-Based Encryption

Attribute-Based Encryption, as the name suggest, is a type of encryption that enforce an access control via attributes and authorization policies. The secret key of a user and the ciphertext are both dependent by attributes.

Proposed in 1984, it was improved and implemented in the first years of 2000s; it presents two main types of encryption schemes: the **key-policy attribute-based encryption (KP-ABE)** [11] and the **ciphertext-policy attribute-based encryption (CP-ABE)** [12].

3.1.1 From Identity-Based to Attribute-Based Encryption

Identity-Based Encryption is a type of public key cryptography based on PKI (Public-Key Infrastructure), with the main difference being the fact that IBE allows a 1 to 1 relationship between sender and reciever. As Shamir said in his paper [13]:

The scheme is based on a public key cryptosystem with an extra twist: Instead of generating a random pair of public/secret keys and publishing one of these keys, the user chooses his name and network address as his public key. Any combination of name, social security number, street address, office number or telephone number can be used (depending on the context) provided that it uniquely identifies the user in a way he cannot later deny, and that it is readily available to the other party.

The signature scheme abide the following verification condition [13]:

$$s^e = i \cdot t^{f(t,m)} \mod n \quad (1)$$

Where

- m is the message,
- s, t is the signature,
- i is the user's identity,
- n is the product of large primes,

- e is a large prime relatively prime to $\phi(n)$,
- f is a one way function.

IBE applications are quite immediate to understand, for example, the ability to ensure that only a given person could read a message, without relying on the PKI, is quite remarkable. There is, anyway, a trust assumption towards the Private Key Generator (PKG), which if compromised would nullify the secrecy of every encrypted data that use its issued keys.

Besides, IBE did point towards ABE; we can see how Sahai e Waters already introduced an ABE approach while proposing a Fuzzy Identity-Based Encryption. They did note how a system where an authority gave a user different keys for each of the attributes that describe the user could easily fall prey of collusion attacks, where multiple users combine their keys to form identities, with a combination of said attributes, that could decrypt ciphertext which none of the individual user could be able to decrypt alone [14]. This occurs by considering an identity as a set of attributes and, precisely for that, FIBE for the first time achieve error tolerance; it allows for a private key for an identity, say ω , to decrypt a ciphertext encrypted with an identity ω' if and only if $|\omega \cap \omega'| \geq d$, where d is fixed during the setup and indicates the error tolerance threshold.

Goyal et al., later proposed a better implementation that was further expanded by Bethencourt, Sahai e Waters, de facto introducing, respectively, KP-ABE and CP-ABE.

3.1.2 Ciphertext-Policy versus Key-Policy ABE

The main difference between KP and CP ABE is their approach to the policy needed to decrypt. While in the Key-Policy ABE the key holds the policy for the identity, in the Ciphertext-Policy ABE is the ciphertext that holds said policy. In other words, for a KP approach, think of a company that encrypt all its mails with the headers as attributes, say `{reciever: developers, object: project-x, date: 20-dec}`. When an analyst, for a given reason, has to read those mails it will receive a new key suited for the occasion, for example:

```
(reciever=developers AND object=project-x) OR (date>20-dec)
```

This key and this key alone can decrypt the mails that satisfy the key's rule.

Using CP ABE, instead, shift the policy on the document itself: always in a corporate environment, the company encrypted a reserved memo already knowing who can read it,

thus it encrypted the document with a policy like:

(department=finance) OR (management AND security_level > 5)

With this setup only those employees who are in the finance department or are in management, with a security level greater than 5, can access this document. In summa, the paradigm shift from one-for-all to all-for-one.

Both KP and CP, however, define a collusion resistant structure and, besides an **Access Structure**, four fundamental algorithm: **Setup**, **Encrypt**, **Key Generation** and **Decrypt** [11, 12]. More specifically, those implementations differ on **Encrypt**, **Key Generation** and **Decrypt**, furthermore, CP ABE includes also a **Delegate** algorithm. It is all defined as:

- **Access Structure:** Let $P_1, P_2, \dots, P_n$ be a set of parties. A collection $A \subseteq 2^{P_1, P_2, \dots, P_n}$ is monotone if $\forall B, C : \text{if } B \in A \text{ and } B \subseteq C \text{ then } C \in A$. An **Access Structure** (monotone access structure) is a collection (monotone collection) A of non-empty subsets of $P_1, P_2, \dots, P_n$, i.e. $A \subseteq 2^{P_1, P_2, \dots, P_n} \setminus \emptyset$. The sets in A are called the authorized sets, and the sets not in A are called the unauthorized sets.
- **Setup:** takes no input other than the implicit security parameter. It outputs the public parameters PK and a master key MK .
- **KP Encrypt:** a randomized algorithm that takes as input a message m , a set of attributes γ , and the public parameters PK . It outputs the ciphertext E .
- **KP Key Generation:** a randomized algorithm that takes as input an **Access Structure** A , the master key MK and the public parameters PK . It outputs the decryption key D .
- **KP Decryption:** takes as input the ciphertext E , encrypted under the set γ of attributes, the decryption key D for access structure A and the public parameters PK . It outputs the message M if $\gamma \in A$.
- **CP Encrypt:** takes as input the public parameters PK , a message M , and an access structure A over the universe of the attributes. The algorithm encrypt M and produce a ciphertext CT such that only a user that possesses a set of attributes that satisfies the access structure will be able to decrypt the message. It is assumed that the ciphertext implicitly contains A .

- **CP Key Generation:** takes as input the master key MK and a set of attributes S that describe the key. It outputs a private key SK .
- **CP Decryption:** takes as input the public parameters PK , a ciphertext CT , which contains an access policy A , and a private key SK , which is a private key for a set S of attributes. If S satisfies A then the algorithm will decrypt CT and return the message M .
- **CP Delegate:** takes as input a secret key SK for some set of attributes S and a set $\tilde{S} \subset S$. It outputs a secret key SK for the set of attributes $\tilde{S}$.

3.1.3 Multi-authority ABE

3.1.4 Functional Encryption as a Generalization

3.2 Identity and Access Management

3.2.1 OAuth 2.0 and PKCE

3.2.2 Keycloak

3.3 The Model Context Protocol

3.4 Hook-Based Interception in AI Coding Agents

4 System Design

4.1 Architecture Overview

Our system is composed by two main component:

- **server-side components:** the components include Identity and Access Management (**Keycloak**), database (**Postgres**), logic handler (**OpenABE**), MCP server.
- **client-side connectors:** adapter and hook grant connection and control over the AI agent.

The server-side components are designed to be each one independent from the others; we have containerized each one of them with `containerd` and made them available with Rancher Desktop.

Client-side there are two parts essential to the correct flow of the program: there is an adapter for each agent and a hook that is the real intermediary between the server-side and the agent.

4.2 Early Prototyping: File Viewer and .abe format

4.2.1 File Structure

To start approaching the concept, it is imperative to subdivide the problem into smaller ones. The first problem that came to mind was to find a way to have into a single file multiple parts, each with its own encryption. To solve this problem, we did create a JSON structure that could contain each encrypted section of the file, i.e. an array containing a single object per section that has two properties: the policy requested to decipher the ciphertext and the ciphertext itself.

Listing 1: .abe format v1

```
{
  "format": "abe-doc",
  "version": 1,
  "sections": [
    {
      "policy": "livello = x",
      "ciphertext": CIPHERTEXT_Lx
    }
  ]
}
```

```

    },
    {
      "policy": "livello = y",
      "chiphertext": CIPHERTEXT_Ly
    },
    {
      "policy": "livello = z",
      "chiphertext": CIPHERTEXT_Lz
    }
  ]
}

```

It's quite easy to point out that this implementation is very inefficient when the number of sections grows. To optimize the amount of operations needed and the structure of the document itself, it was improved in the version 2 of the structure:

Listing 2: .abe v2

```

{
  "format": "abe-doc",
  "version": 1,
  "sections": [
    {
      "group": 0
    },
    {
      "group": 1
    },
    {
      "group": 2
    },
    {
      "group": 1
    },
    {
      "group": 0
    }
  ],
  "groups": [
    {

```

```

    "policy": "livello >= 3",
    "ciphertext": CIPHERTEXT_L3
  },
  {
    "policy": "livello >= 2",
    "ciphertext": CIPHERTEXT_L2
  },
  {
    "policy": "livello >= 1",
    "ciphertext": CIPHERTEXT_L1
  }
]
}

```

This structure provides one key feature: the ability to call the decrypt function n times, where n is the number of unique attributes used to rules the sections. This is a more sustainable approach than v1, since, in that case, the decrypt function would be called exactly m times, where m is the amount of each section in the document. It's a fair assumption to say that m , *usually*, is greater than n .

4.2.2 File Viewer

Having a structured document by itself was not sufficient for our testing needs and, having the need to read and edit a .abe file, it was necessary to create such application. The file viewer operates as a simple file editor, with the ability to encrypt and decrypt a file using a user's private key (N.B. this feature would negate entirely the Keycloak IAM workflow and was created only for test). This private key, as seen before, has to be created and stored beforehand. A user could also authenticate itself via **Keycloak** login and so a token with its attribute will be elaborated and used in substitution of the user's key. The user-intended workflow is to login with **Keycloak** and only then he'd have access to the decryption of sections to him destined.

There is an implicit security system, that is any user with the "security level" (an attribute defined to test a hierarchy) less than the maximum security level used in the file cannot overwrite the file.

4.3 CP-ABE Authority and Key Management

It is clear that we have to have a central authority that could audit and grant accesses to documents. This operation is made by conjuncting both **Keycloak** and **Postgres**: While **Keycloak** holds every user's informations (e.g. mail, department, security level, etc...), and so its attributes, **Postgres** holds both the users' cached keys, once they received them from Keycloak, and a list of allowed agents with each own key.

The concept is simple: first a user get its token from Keycloak, then a cached and hashed key is kept in memory in a trusted sql database, and then, if in said database is found a key corresponding to the user's agent, the request is accepted by the server who will run the **OpenABE** operations to serve the decrypted file. With this solution nobody needs to have installed its own key and it is strictly handled "in background", to both minimizing the risk of tampering and also to ensure a strict control over the allowed agents (it's easy as deleting a row in a db to disable entirely an agent from being used).

4.4 Human Identity via Keycloak

Obviously, the Keycloak login can and has to be used with two-factor-authentication (it can be enforced also a biometric login). The 2fa itself is out of this scope, but we mention **FreeOTP** as a free tool for an otp testing.

Once the user is logged in, and we can assume it is authenticated, then we can proceed to concede a token, containing the relevant attributes, that will be used to ask for the key of said user.

4.5 Agent Identity Model

4.6 Audit Logging

Each and every operation the agent make while calling the server is logged, both to log every access to each document and to ensure their confidentiality. Server-side it is all kept in a `audit.jsonl`, for semplicity's sake, but it can be enhanced.

4.7 Core and Strategy Pattern

4.7.1 The Relay Hook

4.7.2 Per-Agent Adapters

4.8 Document Lifecycle

The document starts its life whenever created and encrypted by someone, say a manager, that has to include sensible information in said document. Once the appropriate policy is defined and used to encrypt the document, it is stored and it becomes available through request at the MCP server.

4.9 Deployment Architecture

5 Security Analysis

6 Evaluation

7 Conclusions

Conclusions

8 Bibliography

Riferimenti bibliografici

- [1] Siladitya Ray. *Samsung Bans ChatGPT And Other Chatbots For Employees After Sensitive Code Leak*. <https://www.forbes.com/sites/siladityaray/2023/05/02/samsung-bans-chatgpt-and-other-chatbots-for-employees-after-sensitive-code-leak/>. Forbes; ultimo accesso 5 agosto 2026. 2023.
- [2] Garante per la protezione dei dati personali. *ChatGPT, il Garante privacy chiude l'istruttoria. OpenAI dovrà realizzare una campagna informativa di sei mesi e pagare una sanzione di 15 milioni di euro*. <https://www.garanteprivacy.it/home/docweb/-/docweb-display/docweb/10085432>. Provvedimento n. 755 del 2 novembre 2024; ultimo accesso 5 agosto 2026. 2024.
- [3] Anthropic. *Updates to Consumer Terms and Privacy Policy*. <https://www.anthropic.com/news/updates-to-our-consumer-terms>. Ultimo accesso 5 agosto 2026. 2025.
- [4] Zeutro. *OpenABE: An Open Source cryptographic library, with implementation of Attribute-Based Encryption in C/C++*. <https://github.com/zeutro/openabe>. 2020.
- [5] Edoardo Debenedetti et al. «Defeating prompt injections by design». In: *arXiv preprint arXiv:2503.18813* (2025).
- [6] Jung Hee Cheon et al. «Homomorphic encryption for arithmetic of approximate numbers». In: *International conference on the theory and application of cryptology and information security*. Springer. 2017, pp. 409–437.
- [7] Leo De Castro et al. «EncryptedLLM: Privacy-Preserving Large Language Model Inference via GPU-Accelerated Fully Homomorphic Encryption.» In: *ICML*. 2025.
- [8] Ahmad Al Badawi et al. «Openfhe: Open-source fully homomorphic encryption library». In: *proceedings of the 10th workshop on encrypted computing & applied homomorphic cryptography*. 2022, pp. 53–63.

- [9] Tim Geppert et al. «Trusted execution environments: Applications and organizational challenges». In: *Frontiers in Computer Science* 4 (2022), p. 930741.
- [10] Ruidong Han et al. «Automated TEE Adaptation with LLMs: Identifying, Transforming, and Porting Sensitive Functions in Programs». In: *IEEE Transactions on Software Engineering* (2026).
- [11] Vipul Goyal et al. «Attribute-based encryption for fine-grained access control of encrypted data». In: *Proceedings of the 13th ACM conference on Computer and communications security*. 2006, pp. 89–98.
- [12] John Bethencourt, Amit Sahai e Brent Waters. «Ciphertext-policy attribute-based encryption». In: *2007 IEEE symposium on security and privacy (SP'07)*. IEEE. 2007, pp. 321–334.
- [13] Adi Shamir. «Identity-based cryptosystems and signature schemes». In: *Workshop on the theory and application of cryptographic techniques*. Springer. 1984, pp. 47–53.
- [14] Amit Sahai e Brent Waters. «Fuzzy identity-based encryption». In: *Annual international conference on the theory and applications of cryptographic techniques*. Springer. 2005, pp. 457–473.